

Original software publication

# RSEMM: A dashboard for evaluating research software maturity

Kwabena Ebo Bennin<sup>ID\*</sup>, Bedir Tekinerdogan, Siamak Farshidi

Information Technology Group, Wageningen University and Research, Wageningen, 6706KN, The Netherlands

## ARTICLE INFO

Dataset link: <https://github.com/SiamakFarshidi/RSEMM-WUR/tree/master>, <https://doi.org/10.17632/t2dygzcsyt.2>, <https://ai4rse.nl/RSEMM/>

### Keywords:

Research software engineering  
Education  
Dashboard  
Software evaluation

## ABSTRACT

Research software often lacks standardized frameworks to assess its quality, sustainability, and impact. This paper introduces the *Research Software Evaluation and Maturity Monitor (RSEMM)*, a web-based dashboard that integrates software engineering metrics, FAIRness evaluation, AI/ML classification, code generation detection, and citation tracking. By providing automated maturity scoring and actionable recommendations, RSEMM supports developers, maintainers, and researchers in improving the quality of open research software. RSEMM has been applied to 1,519 open-source research projects, with all evaluation results and metadata openly available on Mendeley Data (<https://doi.org/10.17632/t2dygzcsyt.2>). The dashboard is accessible online at <https://ai4rse.nl/RSEMM/>.

## Code metadata

Current code version  
Permanent link to code/repository  
Legal Code License  
Code versioning system  
Software languages and tools  
Compilation requirements  
Link to dataset (Mendeley Data)  
Developer documentation/manual  
Support email

v1.0  
<https://github.com/ElsevierSoftwareX/SOFTX-D-25-00562>  
GNU General Public License (GPL) v3  
Git  
Python, FastAPI, JavaScript (React), Docker  
Linux or Windows with Docker, Python 3.9+  
<https://doi.org/10.17632/t2dygzcsyt.2>  
<https://ai4rse.nl/docs/RSEMM>  
[siamak.farshidi@wur.nl](mailto:siamak.farshidi@wur.nl)

## 1. Motivation and significance

Research software has become a cornerstone of modern science, yet assessing the maturity, sustainability, and impact of Research Software remains challenging. Research Software Engineering (RSE) represents a rapidly maturing discipline that bridges the gap between traditional research and professional software development, focusing on applying robust engineering principles to the creation, maintenance, and sustainability of scientific code [1]. RSEs possess a unique blend of domain knowledge and software expertise, enabling them to transform prototypes into reliable, reusable, and reproducible research tools, thereby accelerating scientific discovery and ensuring the longevity of computational methods. Existing evaluation methods are fragmented and often require deep technical expertise or manual effort. Research Software Evaluation and Maturity Monitor (RSEMM) addresses this

gap through a modular, automated system that synthesizes diverse evaluation dimensions into a coherent dashboard. Assessing the maturity and sustainability of research software has become increasingly important for transparency, reproducibility, and long-term usability. However, many projects lack structured mechanisms for evaluation. Existing efforts have focused on specific aspects of research software but typically operate in isolation.

The FAIR Principles for Research Software (FAIR4RS) provide a foundational framework, emphasizing findability, accessibility, interoperability, and reusability [2]. Tools like howfairis [3] operationalize FAIR4RS by automatically checking metadata and repository compliance. However, these tools are limited to FAIR aspects and do not consider engineering practices or community impact. OpenAlex [4] provides citation and bibliometric data to understand scholarly influence, but it is not tailored for software repositories. Similarly, software

\* Corresponding author.

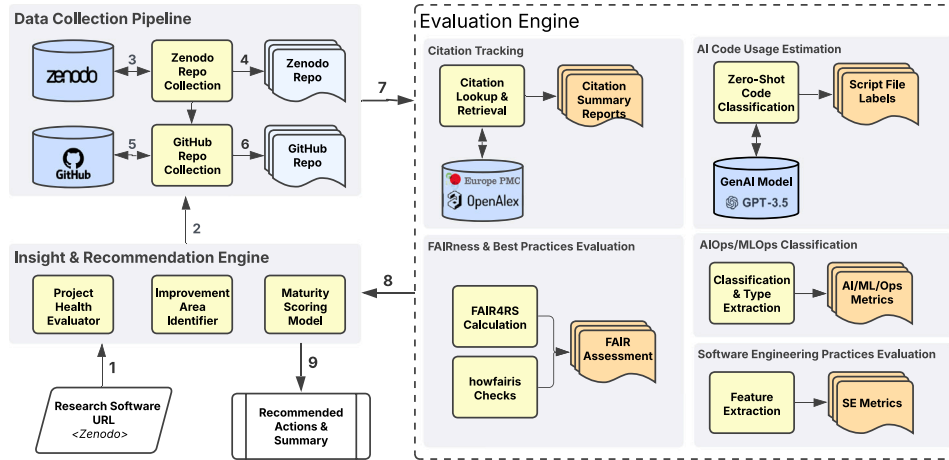
E-mail addresses: [kwabena.bennin@wur.nl](mailto:kwabena.bennin@wur.nl) (K.E. Bennin), [bedir.tekinerdogan@wur.nl](mailto:bedir.tekinerdogan@wur.nl) (B. Tekinerdogan), [siamak.farshidi@wur.nl](mailto:siamak.farshidi@wur.nl) (S. Farshidi).

<https://doi.org/10.1016/j.softx.2025.102437>

Received 6 August 2025; Received in revised form 28 October 2025; Accepted 31 October 2025

Available online 11 November 2025

2352-7110/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).



**Fig. 1.** Architecture of RSEMM: The system collects metadata from GitHub or Zenodo, processes it through multiple evaluation modules, and synthesizes scores and recommendations into a unified dashboard.

engineering tools assess code quality or test coverage (e.g., static analysis linters, CI checkers), but they do not provide holistic maturity feedback.

RSEMM addresses this gap by integrating multiple evaluation dimensions, including FAIRness, software engineering metrics, AI-generated code detection, and citation tracking, into a single system. Unlike siloed tools, RSEMM provides a unified dashboard for research software assessment. It draws inspiration from quality models in software engineering but adapts them to the distinct needs of research codebases, often maintained by academic or citizen scientists with limited resources. RSEMM builds upon and extends these existing efforts by combining metrics from multiple domains into an actionable, interpretable report for developers and researchers.

The paper is organized as follows: Section 2 describes the software, including its architecture and evaluation metrics; Section 3 provides illustrative examples that demonstrate a general overview of the dashboard; Section 4 presents the impact of the dashboard. Conclusion and Future research are presented in Section 5.

## 2. Software description

RSEMM is structured as a unified, multi-layered system designed to process disparate data sources into a single, comprehensive assessment for research software. The aim is to offer a broader perspective, including aspects of AI for RSE, which introduces additional dimensions that need to be evaluated in parallel. It is maintained by the AI4RSE Lab at Wageningen University & Research. The tool provides a flexible framework for evaluating research software and is continually developed to align with evolving community standards and best practices. The current development efforts, including datasets and data analysis, are available via Mendeley Data [5]. An up-and-running version of the tool is accessible online (see Table 1).

### 2.1. Architecture and component workflow

RSEMM is structured as a modular, multi-layered system designed [6] to automate and integrate diverse aspects of research software evaluation. It consists of three architectural layers: data collection, evaluation modules, and recommendation synthesis. Each layer is realized through a coordinated set of functional components (see Fig. 1).

### 2.2. Data collection pipeline

The system begins by ingesting a GitHub repository or Zenodo record. It retrieves metadata, including repository name, topics, descriptions, and DOIs. For GitHub sources, it also pulls README files, commit logs, test directories, and CI configurations. Zenodo records are parsed for FAIR-aligned metadata fields, including author identifiers, licensing information, versioning details, and related resources. GitHub API authentication, pagination, and rate-limiting are handled transparently to ensure robustness.

### 2.3. Evaluation engine

At the core of RSEMM, the evaluation engine coordinates five distinct modules, each producing normalized, interpretable results.

#### 2.3.1. FAIRness and best-practice evaluation

This module assesses compliance with the FAIR4RS sub-principles using Zenodo metadata fields. Each of the 15 sub-principles is scored (yes, partial, no/unknown), grouped into the four FAIR categories, and aggregated into an overall FAIRness level (High/Medium/Low) [2,7]. Additional checks on the GitHub repository, such as the presence of a license, citation file, and quality badge, contribute to a separate best-practices score (0–5 scale).

#### 2.3.2. Citation tracking

This module queries OpenAlex and Europe PMC to retrieve citation counts, co-authorship structures, and publication metadata linked to the software's DOI. The output includes citation velocity, h-index-style measures, and author-network summaries to reflect research impact and scholarly integration [4].

#### 2.3.3. AI/ML workflow classification

To detect integration of AIOps or MLOps practices, this module scans concatenated metadata, README content, and commit logs for workflow-specific keywords. Projects are categorized as AIOps, MLOps, Ambiguous, or None. For ambiguous cases, an LLM resolves the label using a prompt-based JSON return. Fine-grained workflow types (e.g., “anomaly detection”, “model serving”) are extracted to enrich the project profile [8].

### 2.3.4. Generated code detection

To estimate the use of large language models during development, RSEMM applies a zero-shot GPT-based classifier to a subset of recent script files. Each file is labeled “AI” or “Human” and an AI ratio is computed:

$$\text{AI ratio} = \frac{N_{\text{AI}}}{N_{\text{AI}} + N_{\text{Human}}}.$$

The result is classified as Low, Medium, or High AI-generated content, helping identify risks in maintainability and provenance [9].

### 2.3.5. Software engineering metrics

This component evaluates project maturity along seven dimensions: community activity, CI integration, documentation coverage, development history, issue management, licensing, and unit testing. Each dimension is scored using empirically grounded thresholds, adapted from curated GitHub datasets [10]. The output yields a per-dimension classification (Low, Medium, High).

## 2.4. Insight and recommendation engine

This final layer consolidates outputs from all modules. It applies heuristic scoring rules to assign a maturity band (“Early-stage”, “Growing”, or “Mature”) and produces an improvement report. The report highlights deficient dimensions (e.g., missing license, no tests) and recommends targeted actions. This synthesis is designed to be accessible to both technical and non-technical stakeholders, supporting transparency and informed project planning.

## 2.5. Recommendation generation

RSEMM scores software maturity across dimensions and flags areas for improvement (e.g., missing license, weak test coverage). A final maturity band (“early-stage”, “growing”, “mature”) is presented with tailored suggestions.

## 3. Illustrative examples

To demonstrate the practical application of RSEMM, we applied the platform to a total of 1,519 open-source research software projects hosted on Zenodo. The evaluation results, including extracted metadata and computed metrics, are publicly available through Mendeley Data [5]. The corresponding evaluated records can also be accessed interactively via the AI4RSE platform (see Table 1).

RSEMM provides a web interface for research software engineers to assess the maturity of their projects. Users can evaluate a Zenodo-hosted repository by pasting its repository or DOI URL into the input field on the platform’s landing page and clicking the *Estimate Maturity* button. Alternatively, they can search for projects previously evaluated in the “Stored Zenodo Records” section and retrieve results by clicking *Load*. Fig. 2 illustrates this interaction model.

Upon submission, the platform automatically calculates a maturity score based on the retrieved Zenodo metadata and the associated GitHub repository content, where available. As shown in Fig. 3, RSEMM presents targeted recommendations tailored to the evaluated repository. These may include adopting continuous integration workflows to enhance code quality, improving documentation to support usability and onboarding, or implementing unit tests to increase reliability. The platform also suggests incorporating modular design patterns for reusability, leveraging automated code generation for development efficiency, integrating AI features to expand functionality, and maintaining current license information for legal and open-science compliance.

Each evaluated repository is analyzed across four core dimensions: *Software Engineering (SE) Efforts*, *FAIR Assessment*, *Code Generation Evaluation*, and *AI/ML/Ops Readiness*. These dimensions aggregate multiple indicators to generate a comprehensive software maturity profile. For

**Table 1**

Inter-rater agreement among human experts and GenAI models on 400 repositories. Cohen’s  $\kappa$  and percentage agreement are reported. GPT refers to OpenAI’s GPT-4o model, and Gemini refers to Google’s Gemini 1.5 Flash model. Combined scores represent majority voting across annotators.

| Evaluator 1      | Evaluator 2      | Cohen’s $\kappa$ | % Agreement |
|------------------|------------------|------------------|-------------|
| Human 1          | Human 2          | <b>0.973</b>     | <b>98.0</b> |
| GPT-4o           | Gemini 1.5 Flash | 0.484            | 69.5        |
| GPT-4o           | Human 1          | 0.379            | 54.0        |
| GPT-4o           | Human 2          | 0.373            | 53.5        |
| Gemini 1.5 Flash | Human 1          | 0.174            | 39.3        |
| Gemini 1.5 Flash | Human 2          | 0.157            | 37.8        |
| Combined GenAI   | Combined Human   | 0.276            | 49.3        |

instance, SE Efforts include commit history, issue tracking, continuous integration usage, documentation quality, license presence, and unit test coverage. The FAIR assessment evaluates metadata completeness and openness following the FAIR4RS principles. The Code Generation Evaluation dimension detects model-driven development or the use of AI-assisted code generation. In contrast, the AI/ML/Ops dimension examines the presence of machine learning functionality and the deployment workflows associated with it.

To ensure transparency, each component includes an information icon that reveals the underlying evaluation protocol. This allows users to understand how the scores were derived and what specific indicators influenced the result.

As shown in Fig. 4, the platform also generates a machine-readable JSON summary for each evaluated repository. This includes identifiers, source metadata, and detailed scoring for each component. This output supports integration into external research dashboards, longitudinal tracking of project evolution, and meta-analysis across multiple repositories.

The large-scale evaluation revealed common gaps across projects, particularly in areas such as licensing, testing, and documentation. Several research software projects have demonstrated improvement in their maturity levels after implementing the feedback and recommendations generated by RSEMM.

## 4. Evaluation and validation

To assess the reliability of RSEMM’s maturity classifications, we conducted an inter-rater agreement study using a randomly sampled set of 400 repositories from the larger evaluation corpus ( $N = 1,519$ ).<sup>1</sup> Each repository was independently analyzed by two human experts with experience in research software engineering as well as by two generative AI models: OpenAI’s *GPT-4o* and Google’s *Gemini 1.5 Flash*. Annotators were instructed to assess the repositories using the same dimensions considered by RSEMM: (1) software engineering practices (community, continuous integration, documentation, commit history, issue tracking, licensing, unit testing), (2) FAIRness compliance, (3) MLOps/AIOps workflow classification, (4) code generation evaluation, and (5) citations and community metadata. Based on these factors, they assigned an overall maturity band (*Early-stage*, *Growing*, *Mature*).

Agreement between annotators was measured using both percentage agreement and Cohen’s  $\kappa$ , which accounts for chance agreement. The results are summarized in Table 1.

<sup>1</sup> The dataset for this validation, including repository identifiers and annotation results, is openly available via Mendeley Data [5], ensuring transparency and reproducibility. A broader theoretical discussion and extended repository analysis can be found in our companion work [11].

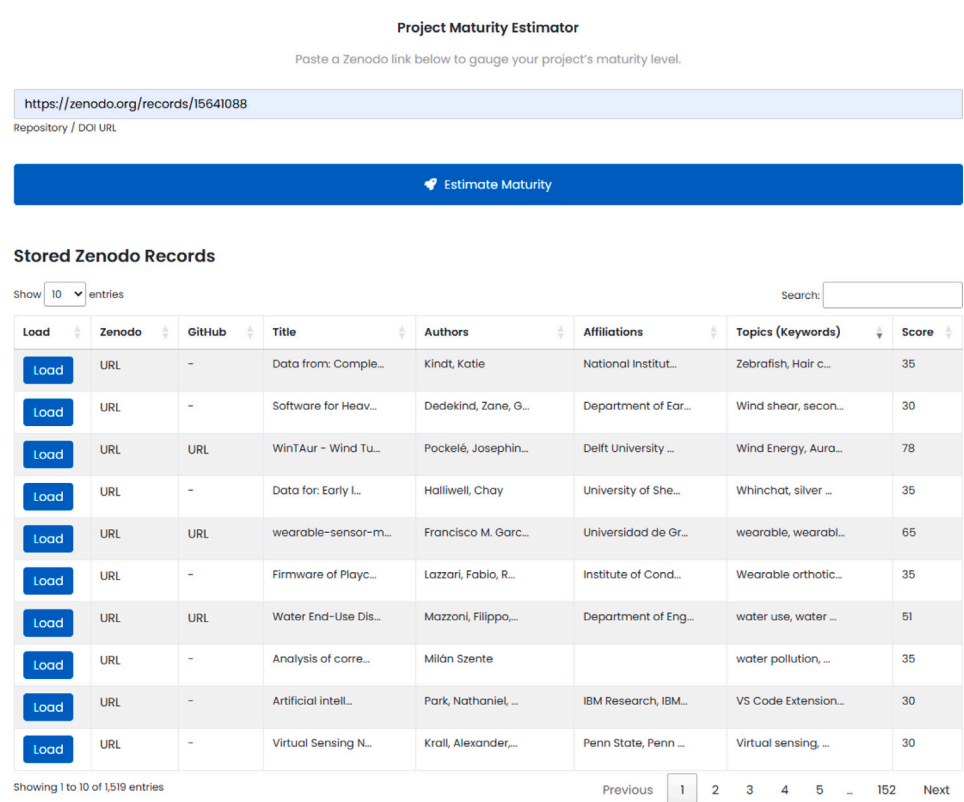


Fig. 2. The RSEMM landing page interface, where users can input a Zenodo URL or browse previously evaluated repositories.

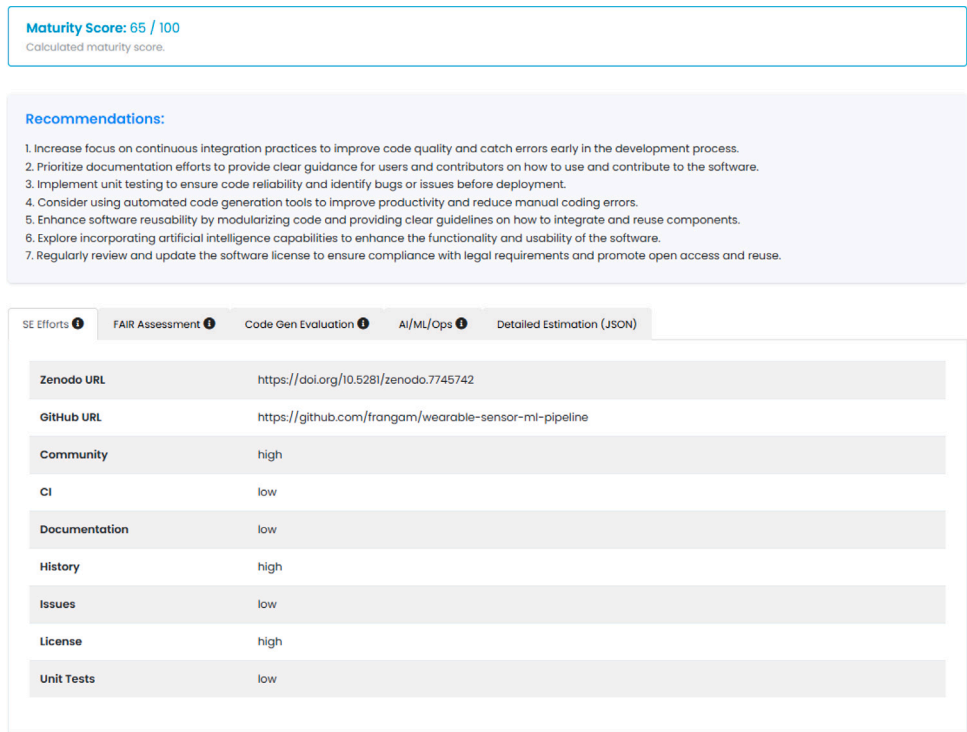


Fig. 3. Example maturity score with automatically generated recommendations to improve software quality and sustainability.

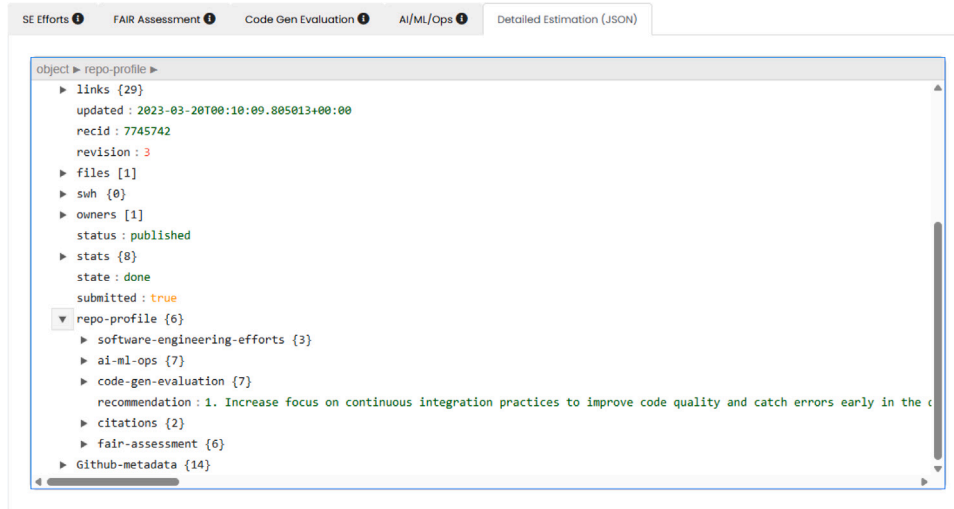


Fig. 4. Detailed metadata summary and evaluation breakdown, including Zenodo and GitHub links and RSEMM component scores.

## Results and analysis

The two human experts demonstrated near-perfect agreement (98%,  $\kappa = 0.973$ ), confirming that the maturity bands used by RSEMM are consistently interpretable by domain specialists. This provides a strong gold standard for evaluating automated methods. In contrast, GPT-4o achieved only moderate alignment with the human annotators ( $\kappa \approx 0.38$ ), while Gemini 1.5 Flash showed weaker agreement ( $\kappa \approx 0.17$ ). Although GPT-4o and Gemini 1.5 Flash agreed with each other more strongly ( $\kappa \approx 0.48$ ), their combined consensus diverged from the human consensus ( $\kappa \approx 0.28$ ).

These discrepancies highlight why general-purpose GenAI models struggled with the task. The assessment required reasoning across multiple dimensions of software quality and reproducibility, including FAIRness, software engineering practices, provenance of code contributions, and community health. These criteria cannot be reliably inferred from surface-level text analysis alone. GenAI models are not explicitly trained for such structured evaluations, and their judgments are sensitive to prompt design and metadata quality. In contrast, human experts can integrate contextual knowledge and domain-specific judgment to resolve ambiguities.

### Accuracy of RSEMM

To quantify the accuracy of RSEMM itself, we compared its maturity band classifications with the human consensus on the same 400 repositories. Out of 392 repositories where both human evaluators agreed, RSEMM achieved an overall accuracy of 89.5%. Table 2 reports precision, recall, and F1-scores for each maturity band. RSEMM demonstrated high precision for *Mature* projects (0.96) and *Early-stage* projects (0.88), as well as strong recall for *Growing* and *Early-stage* projects (above 0.95). The slightly lower recall for *Mature projects with intensive AI practices* (0.81) suggests that RSEMM occasionally underestimates maturity when AI-related activities are not explicitly documented.

These results demonstrate that RSEMM reproduces expert human judgments with high accuracy across the maturity bands. While some disagreement remains in borderline cases, particularly for projects that employ partial or implicit AI practices, the tool provides a reliable and reproducible approximation of expert evaluation.

Table 2

Validation of RSEMM against human consensus (n = 392 repositories). Precision, recall, and F1-scores are reported per maturity band, along with overall accuracy.

| Maturity band         | Precision | Recall | F1-score | Support |
|-----------------------|-----------|--------|----------|---------|
| Early-stage           | 0.884     | 0.955  | 0.918    | 88      |
| Growing               | 0.780     | 0.951  | 0.857    | 82      |
| Mature                | 0.959     | 0.894  | 0.925    | 104     |
| Mature (AI-intensive) | 0.960     | 0.814  | 0.881    | 118     |
| Overall accuracy      | 0.895     |        |          |         |

## 5. Impact

The RSEMM dashboard offers several key impacts for software developers, particularly those working with research codebases. In the following, we present the potentially significant impact of RSEMM on the software engineering community.

**Holistic View of Code Quality Assessment:** RSEMM moves beyond single-metric tools by combining FAIRness (Findable, Accessible, Interoperable, Reusable) principles, software engineering metrics, AI-generated code detection, and citation tracking. For research software developers, this implies comprehensive insight. Instead of juggling separate tools for code quality, reusability, or adherence to open science principles, developers get a unified view. This enables them to quickly grasp the overall health and impact of their codebase. In addition, this dashboard ensures that problems are detected early. By integrating diverse metrics, potential issues related to maintainability, sustainability, or even the ethical implications of AI-generated code can be identified earlier in the development cycle, reducing technical debt and refactoring efforts down the line.

**Actionable and Interpretable Reports:** The dashboard's focus on "actionable, interpretable reports" directly benefits developers by providing clear guidance. Developers are not only presented with raw data; RSEMM aims to provide insights that directly suggest improvements. For example, if a FAIRness metric is low, the dashboard might highlight specific areas of documentation or metadata that need attention. Furthermore, the report presented by the dashboard reduces analysis paralysis. With multiple siloed tools, developers spend considerable time interpreting disparate reports and manually correlating findings.

RSEMM streamlines this, allowing developers to spend more time coding and less time analyzing.

**Tailored for Research Codebases:** The adaptation of quality models of RSEMM to meet the distinct needs of research codebases and academic or citizen scientists with limited resources is crucial. We leverage existing relevant metrics from the literature and adapt them to the needs of research scientists. Traditional software engineering metrics may not always align with the often experimental and evolving nature of research code. The tailored approach of RSEMM ensures that the evaluations are meaningful and relevant to the unique challenges faced by researchers. RSEMM also ensures that resources are optimized: For developers in academic or citizen science settings, who often lack dedicated QA teams or extensive budgets, a unified system reduces the overhead of managing multiple tools and provides essential insights without requiring deep expertise in each evaluation dimension. This helps them make the most of their limited time and resources.

**Enhanced Collaboration and Visibility:** A unified dashboard inherently fosters better collaboration and increases code visibility. RSEMM provides a shared understanding of the software codebase amongst stakeholders. All stakeholders (developers, researchers, project managers, and others) can assess the same comprehensive report, leading to a shared understanding of the status and priorities of the codebase. Finally, citation tracking, in particular, allows developers and researchers to see the real-world impact of their software, potentially motivating them and justifying further development or maintenance efforts. This also helps to demonstrate the value of their work to funders or institutions.

## 6. Threats to validity

The limitations of the RSEMM tool arise from the deliberate selection of evaluation modules, which prioritize automated scalable metrics (FAIRness, basic software quality) over resource-intensive alternatives such as deep semantic code analysis or qualitative assessments centered on humans (UX, community sentiment).

**Internal Validity:** The simplified metrics cannot guarantee true code quality, potentially overlooking subtle security or performance issues.

**External Validity:** The scoring may not be generalizable to industry-grade software, and fixed weighting could misrepresent the sustainability of the project to external sponsors.

**Construct Validity:** Modules such as AI-code detection are prone to technological obsolescence and inaccuracy, risking unfair evaluation of code integrity.

## 7. Conclusion and future work

RSEMM demonstrates the value of an integrated approach to evaluating research software. RSEMM is an innovative dashboard designed to bridge the gap in the evaluation of research software by offering a unified system that integrates multiple crucial evaluation dimensions. Unlike fragmented, siloed tools, it provides developers and researchers with an actionable, interpretable report by combining FAIRness principles, software engineering metrics, AI-generated code detection, and citation tracking. This approach adapts traditional software quality models to the unique needs of research codebases, often maintained by academic or citizen scientists with limited resources, ultimately enabling them to produce higher-quality, more impactful research software. Future plans include tracking reproducibility, measuring team collaboration metrics, and deploying the API as an open platform for community use.

## CRedit authorship contribution statement

**Kwabena Ebo Bennin:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Methodology, Investigation. **Bedir Tekinerdogan:** Writing – review & editing, Writing – original draft, Supervision, Project administration. **Siamak Farshidi:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgments

This work was conducted as part of the AI4RSE Lab's initiatives<sup>2</sup> within the Information Technology Group at Wageningen University & Research. The authors would like to thank all members of the AI4RSE Lab team whose insights helped shape the idea behind this tool. We gratefully acknowledge the FAIR4RS Working Group, the OpenAlex team, and the broader open science communities whose efforts and contributions were instrumental to this research.

## Data availability

RSEMM, including its source code and evaluation framework, is openly available at <https://github.com/SiamakFarshidi/RSEMM-WUR/tree/master>. All datasets, evaluation results for 1,519 research software projects, and supporting data analyses are hosted on Mendeley Data [5] (<https://doi.org/10.17632/t2dygzcsyt.2>). The running platform can be accessed at <https://ai4rse.nl/RSEMM/>.

## References

- [1] Felderer M, Goedicke M, Grunske L, Hasselbring W, Lamprecht A-L, Rumpe B. Investigating research software engineering: Toward RSE research. *Commun ACM* 2025;68(2):20–3.
- [2] Chue Hong NP, Katz DS, Barker M, Lamprecht A-L, Martinez C, Psomopoulos FE, Harrow J, Castro LJ, Gruenpeter M, Martinez PA, et al. FAIR principles for research software (FAIR4RS principles). 2022, Zenodo.
- [3] Spaaks JH, Martinez-Ortiz C. Howfairis: A tool to evaluate the FAIRness of research software. 2020, version 0.13.2. <https://github.com/fair-software/howfairis>.
- [4] Priem Jason, Piwowar Heather, Orr Richard. Openalex: a fully-open index of scholarly works, authors, venues, institutions, and concepts. *arXiv preprint arXiv:2205.01833* 2022.
- [5] Farshidi S. AI4RSE. 2025, <http://dx.doi.org/10.17632/t2dygzcsyt.2>.
- [6] Dobrica L, Niemela E. A survey on software architecture analysis methods. *IEEE Trans Softw Eng* 2002;28(7):638–53.
- [7] Lamprecht A-L, Garcia L, Kuzak M, Martinez C, Arcila R, Martin Del Pico E, Dominguez Del Angel V, Van De Sandt S, Ison J, Martinez PA, et al. Towards FAIR principles for research software. *Data Sci* 2020;3(1):37–59.
- [8] Zarour M, Alzabut H, Alsarayrah K. MLOps best practices, challenges and maturity models: A systematic literature review. *Inf Softw Technol* 2025;107733.
- [9] Yang X, Zhang K, Chen H, Petzold L, Wang WY, Cheng W. Zero-shot detection of machine-generated codes. 2023, *arXiv preprint arXiv:2310.05103*.
- [10] Munaiah N, Kroh S, Cabrey C, Nagappan M. Curating GitHub for engineered software projects. *Empir Softw Eng* 2017;22:3219–53.
- [11] Farshidi S, Bennin K, Babur Ö, Sallou J, Kassahun A, Tekinerdogan B. Advancing research software engineering with AI: A research framework. 2025, <http://dx.doi.org/10.21203/rs.3.rs-7178452/v1>, preprint (Version 1) available at Research Square.

<sup>2</sup> <https://ai4rse.nl/>